

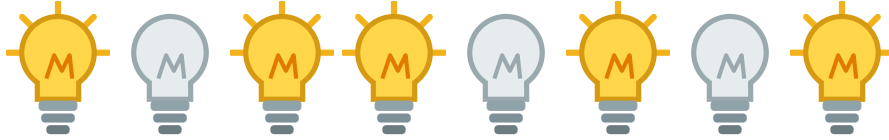
Name: _____

Class: _____

Date: _____

Bits & Bytes Handbook

How computers store everything with 0s and 1s



How to use this handbook

Each lesson page explains one idea with pictures and worked examples. Flip the page over for practice. Light bulbs show bits: a bulb that's **ON** is a 1, a bulb that's **OFF** is a 0.

What's inside

1. What is a bit?
2. From bits to bytes
3. Binary → decimal
4. Decimal → binary (step by step)
5. Counting, limits & overflow
6. Bits can mean anything
7. Hexadecimal basics **bonus**
8. Hex conversions & colors **bonus**

The two bulbs to remember



1 = ON



0 = OFF



Think:

Every photo, song, text message, and video game on your phone is really just a giant pattern of ON and OFF switches.

Name:

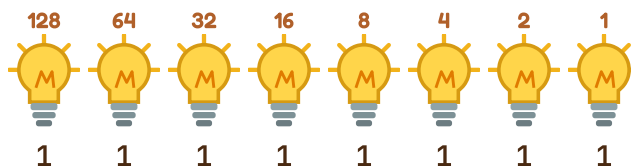
Class:

Date:

Quick Reference

Keep this page handy

8-bit place values (each bulb is worth double the one to its right)



$$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

Powers of two

Bits (n)	Patterns (2 ⁿ)	Largest number (2 ⁿ - 1)
1	2	1
2	4	3
3	8	7
4	16	15
5	32	31
6	64	63
7	128	127
8	256	255
10	1,024	1,023
16	65,536	65,535

Decimal · binary · hex

Dec	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Binary → decimal

Add up the place values of every bulb that's ON.

Decimal → binary

Start with the largest place value. If it fits, write 1 and subtract; if not, write 0. Move right.

Lesson 1 · What Is a Bit?

The smallest piece of data



The Big Idea

Computers store EVERYTHING using only two symbols: 0 and 1. Each single 0 or 1 is called a **bit**, short for **binary digit**.

One bulb = one bit



1

ON



0

OFF

A bit has exactly **two** possible states. We can call them:

- on / off
- 1 / 0
- yes / no
- true / false

Why only two states?

Inside every computer chip are **billions of tiny switches** called **transistors**.

Each switch is either letting electricity flow (**on = 1**) or not (**off = 0**).

Two states are easy to build and hard to confuse — even if the electricity is a little noisy, “on” and “off” are easy to tell apart.

Bits team up

One bulb can only send two messages. But line up several bulbs and you get many different **patterns**:



1

0

1

1

0

This row of 5 bits reads **10110**

Computers read bits in groups — and the more bits you have, the more things you can represent. That's the next lesson!



Think:

A bit is like a light switch: it can only be up or down. Everything amazing a computer does comes from combining LOTS of these simple switches.

Name: _____

Class: _____

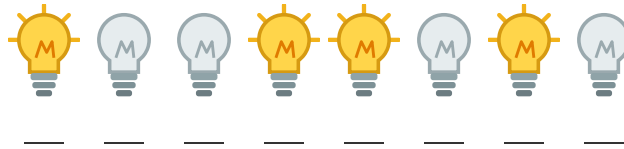
Date: _____

Practice 1

What is a bit?

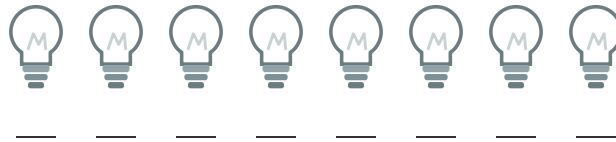
A • Read the bulbs

Write the bit (1 or 0) under each bulb.



B • Light the bulbs

Shade (color in) the bulbs that should be ON to show 01101101.



C • Two states

A bit can be stored by anything with two states. Name four pairs of opposite states (example: on / off).

D • True or false?

1. A bit can be 0, 1, or 2. _____
2. “Bit” is short for “binary digit.” _____
3. Transistors act like tiny on/off switches.

4. One bit can show four different messages. _____

E • Explain it

Why do computers use just two states (0 and 1) instead of ten digits like we do?

Lesson 2 · From Bits to Bytes

How many patterns can bits make?



The Big Idea

Every bulb you add **doubles** the number of patterns. With n bits you can make 2^n different patterns.

1 bulb



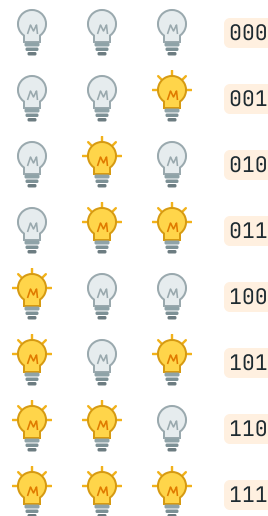
2 patterns

2 bulbs



4 patterns

3 bulbs



8 patterns

The doubling rule

Bits	Patterns
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256

8 bits = **256** patterns, so one byte can store any number from **0 to 255**.

Bits, bytes & beyond

Unit	Size
Bit	one 0 or 1
Nibble	4 bits
Byte	8 bits
Kilobyte (KB)	about 1,000 bytes
Megabyte (MB)	about 1,000,000 bytes
Gigabyte (GB)	about 1,000,000,000 bytes
Terabyte (TB)	about 1,000,000,000,000 bytes

Some systems use 1,024 instead of 1,000 for KB, MB, etc. Either way: each step is about 1,000 times bigger.



Think:

A **byte** (8 bits) is the standard “chunk” of data — about enough to store one letter of text.

Name: _____

Class: _____

Date: _____

Practice 2

Bits to bytes

A · All 3-bit patterns

Shade the bulbs to show all 8 different patterns of 3 bits (no repeats!), and write each pattern.

1.    _____
2.    _____
3.    _____
4.    _____
5.    _____
6.    _____
7.    _____
8.    _____

B · Keep doubling

Bits	Patterns
1	2
2	4
3	
4	
5	
6	
7	
8	
9	
10	

C · Unit math

1. How many bits are in 3 bytes? _____
2. How many bits are in a nibble? _____
3. A text message is 160 bytes. How many bits is that? _____
4. Put in order from smallest to largest: GB, bit, KB, byte, MB, TB

5. About how many bytes are in 5 MB? _____

D · Explain it

Why does adding just ONE more bit double the number of patterns?

Name: _____

Class: _____

Date: _____

Lesson 3 · Binary → Decimal

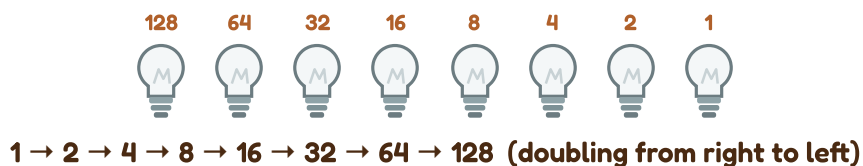
Reading the bulbs



The Big Idea

Each bulb has a **place value**. The rightmost is worth 1, and each bulb to the left is worth **double**. To read a binary number, add up the values of the bulbs that are ON.

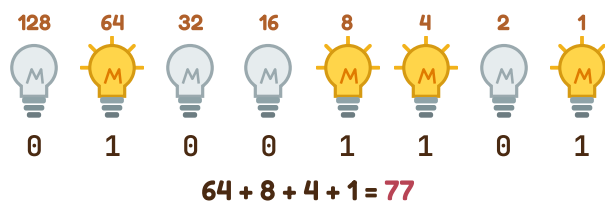
The place values



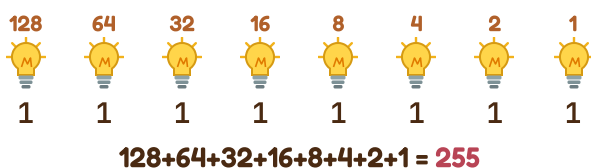
Step by step

- 1 Write the place values (128, 64, 32, 16, 8, 4, 2, 1) above the bits, lined up from the **right**.
- 2 Find every bulb that's **ON** (every 1).
- 3 **Add** their place values. Ignore the OFF bulbs.
- 4 That total is your decimal number.

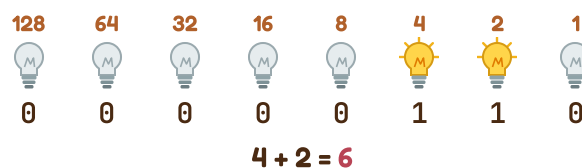
Example: 01001101



Example: 1111111



Example: 00000110



Think:

Two shortcuts: leading zeros don't change the value (0101 = 101 = 5), and if the rightmost bit is 1 the number is odd.

Name: _____

Class: _____

Date: _____

Practice 3

Binary → decimal

A · Read the bulbs

	128	64	32	16	8	4	2	1	
1.									= _____
	0	1	0	1	1	0	1	0	
2.									= _____
	0	0	0	1	0	0	1	1	
3.									= _____
	1	0	0	0	0	0	0	1	
4.									= _____
	1	1	0	0	0	0	0	0	

B · Convert

Binary	Decimal
1010	
11001	
01100100	
10000000	
00111111	
11110000	
10101010	
01010101	

C · Quick thinking

1. Is 1011011 odd or even? How can you tell without converting? _____
2. What is the largest number you can make with 6 bits? _____
3. Which is bigger: 1000 or 0111? _____
4. What happens to a binary number's value when you add a 0 on the right end? (Try 101 → 1010.) _____

D · Explain it

Explain to a friend how to convert 00101100 to decimal, step by step.

Name: _____

Class: _____

Date: _____

Lesson 4 · Decimal → Binary

Start with the **LARGEST** place value



The Big Idea

To turn a number into binary, start with the biggest bulb and work right. If the bulb's value fits into what's left, turn it ON and subtract. If not, leave it OFF.

Step by step

- 1 Write the place values: **128 64 32 16 8 4 2 1**.
- 2 Start at the **left** — the largest place value.
- 3 Ask: "Does this value fit into my number (is it $\leq$ what's left)?"
- 4 **YES** → turn the bulb ON (write 1) and **subtract** it.
NO → leave the bulb OFF (write 0).
- 5 Move one place to the right and ask again with what's left.
- 6 When you reach 0, every remaining bulb is OFF.
- 7 **Check:** add up the ON values — you should get your starting number.

Example: 45

Place value	Fits in what's left?	Bit	Left over
128	no	0	45
64	no	0	45
32	YES	1	13
16	no	0	13
8	YES	1	5
4	YES	1	1
2	no	0	1
1	YES	1	0



$$45 = 00101101$$

Check: $32 + 8 + 4 + 1 = 45$ ✓

Example: 200

Place value	Fits in what's left?	Bit	Left over
128	YES	1	72
64	YES	1	8
32	no	0	8
16	no	0	8
8	YES	1	0
4	no	0	0
2	no	0	0
1	no	0	0



$$200 = 11001000$$

Check: $128 + 64 + 8 = 200$ ✓



Think:

Think of it like paying with bills: to pay \$45 with \$128, \$64, \$32, \$16, \$8, \$4, \$2, and \$1 bills (one of each), always grab the **biggest bill that still fits first**.

Name: _____

Class: _____

Date: _____

Practice 4

Decimal → binary

A · Light the bulbs

Shade the bulbs that are ON, then write the 8-bit binary number.

	128	64	32	16	8	4	2	1	
9									= _____
27									= _____
50									= _____
99									= _____
128									= _____
173									= _____

B · Use the step table for 150

Place	Fits?	Bit	Left
128			
64			
32			
16			
8			
4			
2			
1			

150 = _____

C · Your turn

Convert to binary (show your work):

7 = _____

255 = _____

64 = _____

100 = _____

Your age = _____

Your birth month (1-12) = _____

D · Challenge

Convert 233 to binary using the “start with the largest” steps. Then check your answer by adding the ON place values back up.

Lesson 5 · Counting, Limits & Overflow

What happens when you run out of bulbs?



The Big Idea

Binary counting works just like a car's odometer: when a place fills up, it flips back to 0 and carries 1 to the left. A fixed number of bits can only count so high.

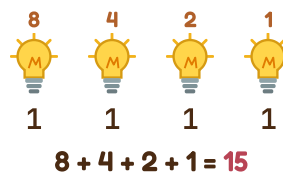
Counting 0 to 15 in 4 bits

Dec	Binary		
0	0000	7	0111
1	0001	8	1000
2	0010	9	1001
3	0011	10	1010
4	0100	11	1011
5	0101	12	1100
6	0110	13	1101
		14	1110
		15	1111

Pattern: the rightmost bit flips every count, the next bit every 2 counts, the next every 4...

The largest number

All bulbs ON = the biggest number the bits can hold:



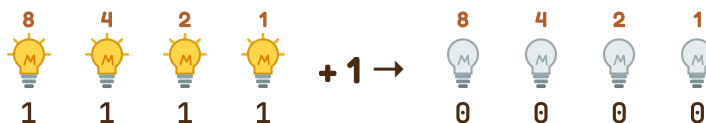
Rule: largest number with n bits = $2^n - 1$.

How many bits do I need?

Keep doubling until you have enough patterns:
Need 100 different values? 2, 4, 8, 16, 32, 64,
128 → 7 bits.

Overflow: out of bulbs!

A 4-bit counter is at 15. Add 1:



The real answer is $16 = 10000$, which needs **5** bits. With only 4 bits, the extra 1 falls off and the counter shows **0**. This is an **overflow error**.



Think:

Overflow happens in real life! Old video games and clocks have “rolled over” when a number got bigger than its bits could hold.

Name: _____

Class: _____

Date: _____

Practice 5

Counting, limits & overflow

A · Keep counting

Continue counting in binary:

Dec	Binary
16	
17	
18	
19	
20	
21	

B · Largest number

Bits	Largest number
3	
5	
6	
8	
10	

C · How many bits?

Find the fewest bits needed for a unique pattern for each:

1. 26 letters of the alphabet _____
2. 100 students _____
3. 300 seats in an auditorium _____
4. 1,000 parking spaces _____
5. Just “yes” or “no” _____

D · Overflow or not?

1. A 4-bit number: $9 + 5$. Overflow? _____
2. A 4-bit number: $12 + 6$. Overflow? _____ What would it show? _____
3. An 8-bit number: $250 + 10$. Overflow? _____
4. An 8-bit number: $100 + 100$. Overflow? _____

E · Explain it

A game stores your score in 8 bits. You have 255 points and earn 1 more. What happens, and how could the programmers fix it?

Lesson 6 · Bits Can Mean Anything

Numbers, text, colors, pictures, and sound



The Big Idea

Bits are just patterns. What they *mean* depends on how the program uses them — that's called **context**. The same 8 bits can be a number, a letter, or part of a color.

Text

Every character has a number code (ASCII / Unicode). The computer stores the code in binary.

Char	Code	Binary
A	65	01000001
B	66	01000010
C	67	01000011
H	72	01001000
I	73	01001001
T	84	01010100

Lowercase letters have different codes (a = 97). A space is 32.

One pattern, two meanings



As a number: 65 · As text: A

The program decides which meaning to use.

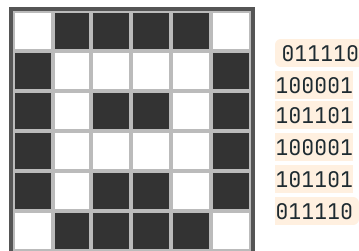
Colors

Screens mix **Red**, **Green**, and **Blue** light. Each is a number from 0 to 255 (one byte), so one color uses **24 bits**.

- (255, 0, 0) = red
- (0, 255, 0) = green
- (255, 255, 0) = yellow
- (255, 255, 255) = white
- (0, 0, 0) = black

Pictures & sound

A picture is a grid of **pixels**. In black and white, each pixel is one bit (1 = black):



Sound is stored by measuring the sound wave thousands of times per second (sampling) and saving each measurement as a number.



Think:

More bits = more detail. More bits per color means more shades; more pixels means a sharper picture; more samples per second means clearer sound — but the file gets bigger.

Date:

Bits can mean anything

Decimal: _____ Message: _____

T = _____

What shape is it? _____

4. How many different colors can 24 bits make? (Hint: 2^{24}) _____

Lesson 7 · Hexadecimal Basics

Base 16: a shortcut for binary



The Big Idea

Binary is long and hard to read. **Hexadecimal** (hex) is a base-16 shorthand: **one hex digit = exactly 4 bits**. Programmers use it for colors, memory addresses, and error codes. **Bonus: hex isn't on the AP CSP exam, but you'll see it everywhere.**

The 16 hex digits

Decimal	Binary (4 bits)	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

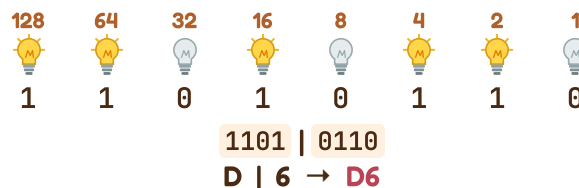
Why letters?

Base 16 needs 16 single symbols. After 0–9 we run out of digits, so we borrow letters:

A=10 B=11 C=12 D=13 E=14 F=15

One byte = two hex digits

Split a byte into two 4-bit halves (nibbles):



Where you'll see hex

- Web colors: #FF8800
 - Device (MAC) addresses: 3C:22:FB:...
 - Error codes and memory addresses: 0x1F
- "0x" or "#" in front tells you a number is hex.



Think:

Base 10 has ten digits (0–9). Base 2 has two (0–1). Base 16 has sixteen (0–F). Same idea, different number of symbols!

Lesson 8 · Hex Conversions & Colors

Step by step, every direction



The Big Idea

To convert with hex, think in groups of 4 bits, or in 16s and 1s. **Bonus topic**

Binary → Hex

- 1 Split the bits into groups of 4, starting from the **right**.
- 2 Add 0s on the left if the last group is short.
- 3 Convert each group to one hex digit.

10111100 → 1011 1100 → **BC**

110101 → 0011 0101 → **35**

Hex → Binary

- 1 Turn each hex digit into its 4-bit pattern.
- 2 Write the groups side by side.

7E → 0111 1110 → 01111110

A5 → 1010 0101 → 10100101

Hex → Decimal (2 digits)

The left digit counts **16s**; the right digit counts **1s**.

$$2F = 2 \times 16 + 15 = 47$$

$$A0 = 10 \times 16 + 0 = 160$$

$$FF = 15 \times 16 + 15 = 255$$

Decimal → Hex

- 1 Divide by 16.
- 2 The whole-number answer is the **left** digit.
- 3 The remainder is the **right** digit.

$$200 \div 16 = 12 \text{ R } 8 \rightarrow \text{C8}$$

$$77 \div 16 = 4 \text{ R } 13 \rightarrow 4D$$

Hex colors: #RRGGBB

Two hex digits each for Red, Green, and Blue (00 = none, FF = full).

 #FF0000 red

 #00FF00 green

 #0000FF blue

 #FFFF00 yellow

 #FFA500 orange

 #800080 purple

 #FFFFFF white

 #000000 black

 #808080 gray



Think:

Every hex color is just 24 bits in disguise: #FF8800 = 11111111 10001000 00000000. Same data, way easier to type!

Date:

Why does #FFFFFF make white? How many different colors can 6 hex digits make?